

INSTITUTO TECNOLÓGICO DE VERACRUZ.

Materia: Administración de Bases de Datos

Alumno: López Ortiz Othón Vladimir

Fecha de entrega: 24 de septiembre de 2026

Trabajo: Documentación de Configuración del Entorno de Trabajo: Activación de WSL, Instalación de Docker Desktop y Verificación de Base de Datos PostgreSQL, Carga BD a Contenedor Docker

1. Resumen.

El presente documento tiene como finalidad reportar los avances técnicos logrados en la preparación e infraestructura del entorno de desarrollo local para el proyecto de administración de bases de datos. Se detallan las etapas de habilitación del Subsistema de Windows para Linux (WSL), la descarga e instalación de Docker Desktop, la verificación y vinculación de la cuenta de usuario con la plataforma Docker Hub y GitHub, así como la inspección de la base de datos de producción local **sira** en PostgreSQL utilizando la herramienta de administración pgAdmin 4. Finalmente, se establece la hoja de ruta para la posterior migración y contenedorización de la base de datos a un entorno aislado basado en Docker.

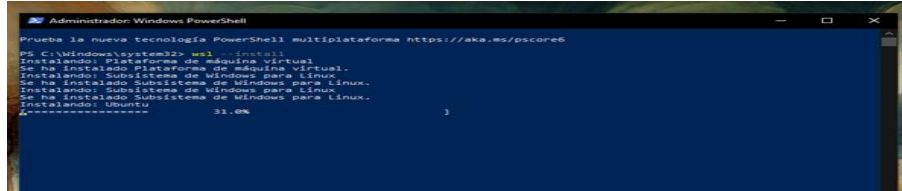
2. Activación e Instalación del Subsistema de Windows para Linux (WSL).

Para lograr un entorno homogéneo y de alto rendimiento compatible con contenedores Linux, se procedió con la instalación de WSL 2 en el sistema operativo host (Windows 10).

El proceso se inició ejecutando la consola Windows PowerShell con privilegios elevados de Administrador. Se ingresó el comando nativo de despliegue automático:

```
wsl --install
```

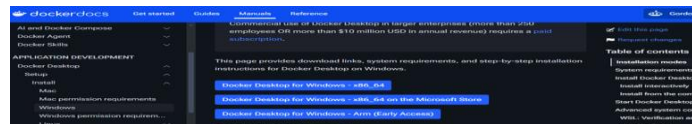
Durante este procedimiento, el sistema operativo realizó la habilitación de la Plataforma de Máquina Virtual, el Subsistema de Windows para Linux y procedió con la descarga de la distribución predeterminada (Ubuntu).



- Figura 1: Ejecución del comando `wsl --install` en Windows PowerShell con privilegios de Administrador, mostrando el inicio de instalación de la Plataforma de máquina virtual y la distribución Ubuntu, Progreso de descarga de la distribución Ubuntu desde el componente ejecutable `wsl.exe` en la terminal.

3. Instalación y Configuración de Docker Desktop

Una vez activado el soporte para WSL 2 y reiniciado el sistema operativo, se procedió a la obtención de Docker Desktop para Windows a través de la documentación y portal oficial de Docker.



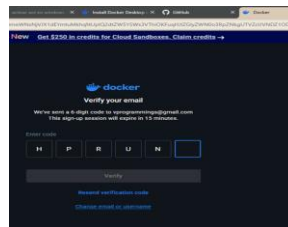
1. **Descarga del instalador:** Se seleccionó el paquete binario de instalación de Docker Desktop para arquitectura `x86_64`.
2. **Ejecución de la instalación:** Se ejecutó el asistente de configuración Docker Desktop Installer.exe. En la ventana de parámetros de configuración, se mantuvo seleccionada la opción recomendada `Per-user installation` y la integración predeterminada con el motor backend de WSL 2.



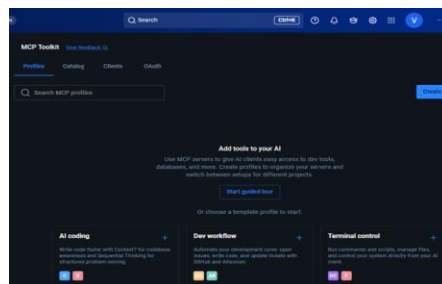
3. **3. Descompresión e instalación de binarios:** El asistente procedió con la extracción de paquetes y la integración de servicios de Docker Desktop.

4. Registro, Verificación de Cuenta y Conexión con Servicios Cloud

Tras completar la instalación de Docker Desktop, se procedió con la verificación y vinculación de credenciales:



1. **1. Verificación de la cuenta Docker:** Se inició sesión en el portal de Docker utilizando la cuenta de correo vprogrammings@gmail.com. Se ingresó el código de verificación de 6 dígitos enviado por correo para activar la sesión de usuario de manera segura.



2. **2. Integración con GitHub:** La cuenta de Docker Hub fue vinculada al perfil de GitHub para permitir en el futuro flujos de trabajo de Integración Continua (CI/CD) y la publicación de imágenes desde repositorios de código fuente.
3. **3. Puesta en marcha del entorno Docker Desktop:** Se verificó el correcto arranque del motor de Docker (Engine running) en el panel principal de la aplicación.

5. Inspección y Verificación de la Base de Datos Local en pgAdmin 4

Para garantizar la integridad de los datos que posteriormente serán migrados hacia un contenedor Docker, se realizó una verificación de los esquemas y registros almacenados localmente en PostgreSQL.

Se ejecutó la herramienta cliente pgAdmin 4, conectándose al servidor de PostgreSQL 18. Dentro de las bases de datos registradas, se inspeccionó la base de datos de producción denominada **sira**, comprobando la presencia de sus tablas, restricciones, funciones y registros listos para la extracción mediante un volcado de datos (dump).

- Figura 9: Ventana de carga inicial (Splash Screen) del entorno de administración pgAdmin 4.
- Figura 10: Interfaz principal de pgAdmin 4 desplegando la estructura del servidor PostgreSQL 18 y el menú de objetos de la base de datos sira.

6. Respaldo y Migración de la Base de Datos SIRA a Docker.

A continuación, se detalla el procedimiento técnico realizado para la extracción, “migración” y restauración de la base de datos de producción sira hacia un entorno aislado de Docker utilizando la imagen oficial de PostgreSQL 18.

6.1. Verificación del usuario de PostgreSQL.

Primero se verificó el usuario utilizado por PostgreSQL desde la terminal de Windows mediante psql. Se ejecutó:

```
psql -U postgres
```

Después de ingresar la contraseña, se comprobó el usuario actual con:

```
SELECT current_user;
```

El resultado obtenido en la consola fue:

```
current_user
```

postgres (1 row)

Por lo tanto, se confirmó que el usuario administrativo de PostgreSQL utilizado era postgres.

Name	Status	Container ID	Image	Created Since	Labels	Ports
postgres	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-1	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-2	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-3	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-4	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-5	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-6	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-7	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-8	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-9	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-10	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-11	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-12	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-13	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-14	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-15	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-16	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-17	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-18	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-19	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-20	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-21	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-22	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-23	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-24	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-25	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-26	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-27	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-28	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-29	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-30	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-31	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-32	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-33	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-34	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-35	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-36	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-37	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-38	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-39	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-40	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-41	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-42	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-43	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-44	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-45	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-46	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-47	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-48	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-49	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-50	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-51	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-52	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-53	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-54	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-55	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-56	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-57	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-58	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-59	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-60	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-61	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-62	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-63	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-64	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-65	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-66	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-67	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-68	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-69	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-70	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-71	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-72	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-73	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-74	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-75	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-76	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-77	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-78	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-79	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-80	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-81	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-82	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-83	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-84	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-85	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-86	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-87	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-88	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-89	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-90	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-91	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-92	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-93	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-94	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-95	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-96	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-97	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-98	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-99	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp
postgres-100	Up 10s	1a1b2c3d4e5f	postgres:18	10 seconds ago		5432/tcp

6.2. Verificación del estado de Docker.

Antes de realizar la migración, se comprobó que Docker estuviera correctamente instalado y funcionando mediante el comando:

`docker info`

La información obtenida en la terminal confirmó que Docker Desktop estaba funcionando correctamente mediante el motor backend de WSL 2. Asimismo, se comprobó inicialmente mediante `docker ps -a` que no existían contenedores ni imágenes activas relacionadas con PostgreSQL.

6.3. Creación e inicialización del contenedor PostgreSQL.

Se creó un nuevo contenedor para PostgreSQL utilizando la imagen oficial `postgres:18`. Se ejecutó el siguiente comando en PowerShell:

```
docker run --name postgres-sira -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=20051309 -e POSTGRES_DB=sira -p 5433:5432 -d postgres:18
```

En este comando se establecieron los siguientes parámetros de despliegue:

--name postgres-sira: Asignó el nombre identificador postgres-sira al contenedor.
-e POSTGRES_USER=postgres: Estableció postgres como el usuario administrador de PostgreSQL.

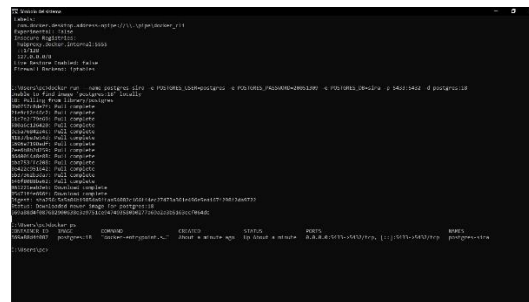
-e POSTGRES_PASSWORD=20051309: Estableció la contraseña del usuario.

-e POSTGRES_DB=sira: Inicializó automáticamente una base de datos vacía llamada sira.

-p 5433:5432: Mapeó el puerto 5433 del sistema host (Windows) con el puerto nativo 5432 de PostgreSQL dentro del contenedor.

-d: Ejecutó el contenedor en segundo plano (detached mode).

postgres:18: Especificó la imagen base de PostgreSQL en su versión 18. Al no encontrarse descargada previamente en el equipo, Docker procedió a descargar sus capas automáticamente desde Docker Hub.



6.4. Verificación del contenedor activo.

Se comprobó que el contenedor estuviera en ejecución mediante:

```
docker ps
```

El resultado en consola mostró:

```
CONTAINER ID IMAGE STATUS PORTS 669a88d4f087 postgres:18 Up About a
minute 0.0.0.0:5433->5432/tcp
```

Esto confirmó que el contenedor postgres-sira estaba activo y que PostgreSQL podía ser accedido desde Windows mediante el puerto redireccionado 5433.

6.5. Generación del respaldo de la base de datos local.

Se realizó un respaldo de la base de datos original siria utilizando la herramienta de volcado pg_dump. Primero se salió de la sesión de psql con \q y desde la consola de comandos (CMD / PowerShell) se ejecutó:

```
pg_dump -U postgres -Fc -d sira -f C:\Users\pc\Desktop\sira.backup
```

Este comando realizó el volcado de la base de datos sira en formato personalizado (-Fc) de PostgreSQL. El archivo generado quedó almacenado en la ruta: C:\Users\pc\Desktop\sira.backup

Se verificó su correcta creación mediante el comando de inspección de directorio: dir C:\Users\pc\Desktop\sira.backup

El archivo generado reportó un tamaño aproximado de 40,817 bytes (~40.8 KB).

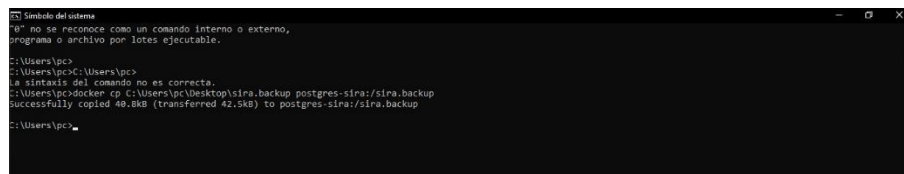
6.6. Copia del archivo de respaldo al contenedor Docker.

Una vez generado el archivo binario de respaldo, se procedió a transferirlo hacia el sistema de archivos del contenedor utilizando el comando docker cp:

```
docker cp C:\Users\pc\Desktop\sira.backup postgres-sira:/sira.backup
```

Docker confirmó la transferencia con el mensaje: Successfully copied 40.8kB ... to postgres-sira:/sira.backup

El archivo quedó almacenado internamente en la raíz del contenedor como /sira.backup.



```
Simbolo del sistema
C:\Users\pc>
C:\Users\pc> docker cp C:\Users\pc\Desktop\sira.backup postgres-sira:/sira.backup
Successfully copied 40.8kB (transferred 42.5kB) to postgres-sira:/sira.backup
C:\Users\pc>
```

6.7. Restauración de la base de datos dentro del contenedor.

Posteriormente, se utilizó la herramienta pg_restore para procesar y restaurar las tablas y datos dentro de la base de datos sira alojada en el contenedor. Se ejecutó:

```
docker exec -it postgres-sira pg_restore -U postgres -d sira /sira.backup
```

El proceso de reconstrucción de objetos y llenado de datos finalizó sin errores, logrando una restauración satisfactoria del esquema de base de datos.

6.8. Verificación de las tablas restauradas.

Para comprobar la integridad de la base de datos migrada, se accedió a la consola interactiva de PostgreSQL dentro del contenedor mediante:

```
docker exec -it postgres-sira psql -U postgres -d sira
```

Una vez iniciada la sesión en la base de datos sira, se ejecutó el comando para listar la relación de tablas:

```
\dt
```

La consulta devolvió un total de 15 tablas pertenecientes al esquema público (public):

El sistema confirmó en la salida (15 rows), verificando que todas las estructuras de la base de datos sira se migraron correctamente.

```
C:\Windows\system - docker exec -it postgres-psql -U postgres -d sira
C:\Users\pc>
C:\Users\pc>C:\Users\pc>
La sintaxis del comando no es correcta.
C:\Users\pc>docker cp C:\Users\pc\Desktop\sira.backup postgres-sira:/sira.backup
Successfully copied 46.8kB (transferred 42.5kB) to postgres-sira:/sira.backup
C:\Users\pc>docker exec -it postgres-sira pg_restore -U postgres -d sira /sira.backup
What's next?
Try Docker Debug for seamless, persistent debugging tools in any container or image - docker debug postgres-sira
Learn more at https://docs.docker.com/go/debug-cli/
C:\Users\pc>docker exec -it postgres-sira psql -U postgres -d sira
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

sira=# \dt
          List of tables
  Schema | Name      | Type  | Owner
  ---
 public | actividad_catalogo | table | postgres
 public | actividad_grupo   | table | postgres
 public | administrador     | table | postgres
 public | alumno            | table | postgres
 public | bonus             | table | postgres
 public | carrera           | table | postgres
 public | configuracion      | table | postgres
 public | estado_unidad     | table | postgres
 public | grupo             | table | postgres
 public | inscripcion        | table | postgres
 public | maestro           | table | postgres
 public | materia           | table | postgres
 public | resultado          | table | postgres
 public | unidad            | table | postgres
 public | usuario           | table | postgres
(15 rows)

sira=#
```

